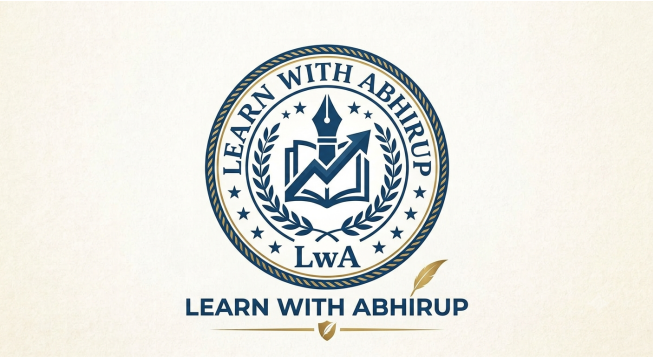
LINUX FROM A TO Z

THE COMPLETE LINUX MASTERY COURSE

From Absolute Beginner to
Confident System Administrator

A Complete Reference & Hands-On Course Covering:

Foundations · Command Line · Users & Processes · Shell Scripting
Networking · System Administration · Security · Performance & Containers



2026 Edition

”The best way to learn Linux is to never stop typing commands into a terminal.”

Contents

PART I

Welcome to the Course

Welcome to **The Complete Linux Mastery Course** — a single reference designed to take you from never having touched a terminal to confidently administering production Linux servers.

This course is organized into progressive parts:

- **Part I — Foundations**: what Linux is, how it came to be, installing it, and the filesystem layout.
- **Part II — Command Line Mastery**: navigating, manipulating files, permissions, and text processing.
- **Part III — Users, Processes & Packages**: managing the system day to day.
- **Part IV — Shell Scripting**: automating everything with Bash.
- **Part V — Networking**: configuring and troubleshooting connectivity.
- **Part VI — System Administration**: systemd, logging, storage, scheduling.
- **Part VII — Security**: hardening, firewalls, SSH, permissions models.
- **Part VIII — Advanced Topics**: performance tuning, kernel basics, containers, virtualization.
- **Part IX — Troubleshooting & Cheat Sheets**: a fast-reference appendix.

Work through it in order if you are new; jump to any chapter if you already have experience. Every chapter includes commands you can type yourself — open a terminal and follow along.

PART II

Part I — Foundations

CHAPTER 1

What Is Linux?

Linux is a free, open-source, Unix-like operating system **kernel** originally created by Linus Torvalds in 1991. Technically, "Linux" refers only to the kernel — the core program that manages hardware (CPU, memory, devices) and provides services to other software. What most people call "Linux" is actually a **distribution** (distro): the Linux kernel bundled with the GNU userland tools (shell, compilers, utilities), a package manager, and often a desktop environment. This is why some people call the full system "GNU/Linux."

1.1 Why Linux Matters

- It powers the majority of the world's web servers, cloud infrastructure (AWS, Azure, GCP all run on Linux), supercomputers, and embedded devices.
- Android, the world's most-used mobile OS, is built on the Linux kernel.
- It is free and open source: anyone can read, modify, and redistribute the source code.
- It is the standard environment for software development, DevOps, cybersecurity, and data engineering.

1.2 Key Characteristics

- **Multiuser**: many users can work on the same system simultaneously, each with isolated permissions.
- **Multitasking**: the kernel schedules many processes to run seemingly at once.
- **Portable**: runs on everything from smartwatches to mainframes.
- **Stable and secure**: a strong permissions model and mature security tooling.
- **Free and open source**: licensed mostly under the GPL.

1.3 Major Distributions

Family	Examples	Package format	Typical use
Debian-based	Debian, Ubuntu, Linux Mint	`.deb` (APT)	Desktops, servers, beginners
Red Hat-based	RHEL, Fedora, CentOS Stream, Rocky/Alma Linux	`.rpm` (DNF/YUM)	Enterprise servers
Arch-based	Arch Linux, Manjaro	`.pkg.tar.zst` (Pacman)	Rolling release, advanced users

Family	Examples	Package format	Typical use
SUSE-based	openSUSE, SLES	`.rpm` (Zypper)	Enterprise
Independent	Alpine, Gentoo, Slackware	varies	Containers, source-based, minimalist

This course uses Debian/Ubuntu examples (`apt`) most often but explains Red Hat equivalents (`dnf`/`yum`) wherever package management is discussed, since these two families cover the vast majority of real-world systems.

CHAPTER 2

A Brief History

- **1969** — Unix is created at Bell Labs by Ken Thompson and Dennis Ritchie.
- **1983** — Richard Stallman launches the GNU Project to build a free Unix-compatible operating system, releasing tools like GCC and Bash but lacking a kernel.
- **1991** — Linus Torvalds, a Finnish student, releases the first Linux kernel as a hobby project and posts about it on Usenet.
- **1992** — Linux is relicensed under the GNU GPL, allowing it to combine with GNU tools to form a complete free operating system.
- **1993–1994** — Debian and Red Hat are founded, formalizing the idea of a "distribution."
- **2004** — Ubuntu launches, dramatically improving ease of use and onboarding new users to Linux.
- **2008** — Android, built on the Linux kernel, ships its first phone.
- Today — Linux dominates servers, cloud computing, and embedded systems, and runs on essentially all of the world's top supercomputers.

CHAPTER 3

Installing Linux

3.1 Ways to Run Linux

1. **Bare metal**: install directly onto a computer, replacing or dual-booting with another OS.
2. **Virtual machine**: run Linux inside software like VirtualBox, VMware, or Hyper-V on top of Windows/macOS — the safest way to learn.
3. **WSL (Windows Subsystem for Linux)**: run a real Linux environment inside Windows without a VM.
4. **Cloud instance**: rent a Linux server from AWS EC2, DigitalOcean, Linode, etc., and access it over SSH.
5. **Live USB**: boot Linux from a USB stick without installing anything, useful for testing.

3.2 General Installation Steps (Ubuntu example)

1. Download the ISO image from the distribution's official website and verify its checksum.
2. Create bootable media using a tool like Rufus (Windows), Etcher, or `dd` (Linux/macOS).
3. Boot from the USB/DVD and choose "Try" or "Install."
4. Configure language, keyboard layout, and timezone.
5. Choose a disk partitioning scheme: use the whole disk (simplest), or manual partitioning for dual-boot or custom layouts (e.g., separate `/home`, `/var`, swap).
6. Create your user account and set a strong password.
7. Wait for installation, reboot, remove installation media.

3.3 Partitioning Basics

A typical manual layout includes:

- `/` (root) — the entire filesystem tree begins here; needs the most space.
- `/boot` — kernel and bootloader files (often 512MB–1GB, can be separate for encryption setups).
- `swap` — virtual memory overflow space, traditionally 1–2x RAM (less critical with modern systems and swap files).
- `/home` — user data; keeping it on a separate partition lets you reinstall the OS without losing personal files.
- `/var` — variable data like logs and databases; sometimes separated on servers

to avoid filling the root partition.

3.4 The Boot Process

Understanding what happens when you power on a Linux machine is essential for troubleshooting:

1. **BIOS/UEFI** — Firmware initializes hardware and looks for a bootable device.
2. **Bootloader (GRUB)** — Loaded from the boot sector or EFI partition; presents a menu (if configured) and loads the selected kernel and initial RAM disk (`initramfs`) into memory.
3. **Kernel initialization** — The kernel decompresses itself, initializes hardware drivers, mounts the root filesystem (initially read-only), and starts the first process.
4. **init system (systemd on most modern distros)** — PID 1 is launched. It reads its configuration and starts all defined targets/services in the correct dependency order (networking, logging, display manager, etc.).
5. **Login** — A getty process presents a login prompt on a virtual console, or a graphical display manager (GDM, SDDM, LightDM) presents a login screen.

Run `systemd-analyze` and `systemd-analyze blame` on a systemd-based system to see exactly how long each step of boot took — invaluable for diagnosing slow boots.

CHAPTER 4

The Linux Filesystem Hierarchy

Unlike Windows, Linux has a single unified tree starting at `/` (root) — there is no concept of "drive letters." Every disk, partition, or device is **mounted** onto a directory within this tree.

4.1 Filesystem Hierarchy Standard (FHS)

Directory	Purpose
`/`	Root of the entire filesystem tree
`/bin`	Essential user command binaries (often symlinked to `/usr/bin` today)
`/sbin`	Essential system binaries for administration
`/boot`	Bootloader files, kernel images, initramfs
`/dev`	Device files representing hardware (e.g., `/dev/sda`, `/dev/null`)
`/etc`	System-wide configuration files
`/home`	Personal directories for each user (`/home/alice`)
`/root`	Home directory of the root (superuser) account
`/lib`, `/lib64`	Shared libraries needed by binaries in `/bin` and `/sbin`
`/media`	Mount points for removable media (USB drives, CDs)
`/mnt`	Temporary mount points for manually mounted filesystems
`/opt`	Optional, third-party software packages
`/proc`	Virtual filesystem exposing kernel and process information
`/sys`	Virtual filesystem exposing kernel/device/driver information
`/run`	Runtime data since last boot (PIDs, sockets)
`/tmp`	Temporary files, usually cleared on reboot
`/usr`	Secondary hierarchy for user-installed software, libraries, docs

Directory	Purpose
`/var`	Variable data: logs, mail spools, caches, databases
`/srv`	Data served by the system, e.g., web or FTP files

4.2 Important Subdirectories

- `/etc/passwd`, `/etc/shadow`, `/etc/group` — user and group account databases.
- `/etc/fstab` — defines filesystems to mount automatically at boot.
- `/var/log` — central location for system and application logs.
- `/usr/bin`, `/usr/sbin`, `/usr/lib` — bulk of installed software.
- `/proc/cpuinfo`, `/proc/meminfo`, `/proc/<pid>/` — live kernel/process data you can `cat` like a normal file.

4.3 Everything Is a File

A foundational Linux philosophy: hardware devices, sockets, pipes, and even kernel data structures are represented as files. This lets you interact with vastly different things using the same small set of tools (`cat`, `read`, `write`). For example:

```
cat /proc/cpuinfo        # view live CPU information
echo 1 > /sys/class/leds/led0/brightness   # control hardware via a "file"
ls /dev                  # list every recognized device
```

4.4 Absolute vs. Relative Paths

- **Absolute path**: starts from root, e.g., `/home/alice/documents/file.txt`.
- **Relative path**: starts from the current directory, e.g., `documents/file.txt` or `../sibling-folder`.
- `.` refers to the current directory; `..` refers to the parent directory; `~` refers to the current user's home directory.

4.5 Mounting and Unmounting

```
lsblk                          # list block devices and their mount points
sudo mount /dev/sdb1 /mnt/usb  # mount a device manually
sudo umount /mnt/usb           # unmount it
df -h                          # show disk usage of mounted filesystems
```

`/etc/fstab` entries follow the format:

```
<device>  <mount point>  <fs type>  <options>  <dump>  <pass>
```

Example:

```
UUID=1234-5678  /home  ext4  defaults  0  2
```

Using `UUID=` instead of `/dev/sdXN` is best practice because device names can shift between boots, while UUIDs stay constant.

PART III

Part II — Command Line Mastery

CHAPTER 5

The Shell and Terminal

The **shell** is the program that interprets commands you type and runs them. The most common shell is **Bash** (Bourne Again SHell), though others exist: `zsh` (popular on macOS and increasingly Linux, with better completion and theming), `fish` (user-friendly, opinionated), `dash` (fast, minimal, used for system scripts), and `ksh`.

The **terminal** (or terminal emulator) is the window/application that hosts the shell. "Terminal," "console," "command line," and "shell" are often used loosely interchangeably in everyday speech.

5.1 Anatomy of the Prompt

A typical prompt looks like:

```
alice@server:~$
```

This shows the username (`alice`), hostname (`server`), current directory (`~` = home), and the prompt symbol (`$` for normal users, `#` for root).

5.2 Getting Help

```
man ls          # full manual page for a command
ls --help       # quick usage summary
whatis ls       # one-line description
info coreutils  # detailed GNU info documents
tldr ls         # community-driven simplified examples (if installed)
```

Inside `man`, press `/word` to search, `n` for next match, `q` to quit.

CHAPTER 6

Navigating the Filesystem

```
pwd                    # print working directory
ls                     # list directory contents
ls -l                  # long listing: permissions, owner, size, date
ls -la                 # include hidden files (dotfiles)
ls -lh                 # human-readable sizes
cd /path/to/dir        # change directory
cd ..                  # go up one level
cd ~                   # go home
cd -                   # go to the previous directory
tree                   # recursive directory tree (if installed)
```

6.1 Wildcards (Globbing)

Pattern	Meaning
`*`	matches any number of characters
`?`	matches exactly one character
`[abc]`	matches one character from the set
`[a-z]`	matches one character in the range
`{txt,md}`	brace expansion: matches either extension

```
ls *.txt             # all .txt files
ls file?.log         # file1.log, file2.log, but not file10.log
ls report_{2024,2025}.csv
```

CHAPTER 7

File and Directory Operations

```
touch newfile.txt          # create an empty file or update its timestamp
mkdir myfolder              # create a directory
mkdir -p a/b/c               # create nested directories at once
cp source.txt dest.txt       # copy a file
cp -r folder1 folder2         # copy a directory recursively
mv old.txt new.txt            # rename or move a file
rm file.txt                    # delete a file
rm -r folder                    # delete a directory recursively
rm -rf folder                    # force-delete, no confirmation (use with extreme
↪  care)
rmdir empty_folder                # remove an empty directory only
ln -s /path/target linkname        # create a symbolic (soft) link
ln /path/target hardlink            # create a hard link
```

Safety tip: `rm -rf /` and similar wide patterns can destroy a system. Modern `rm` refuses to operate on `/` directly without `--no-preserve-root`, but always double check paths before pressing Enter, especially with wildcards.

7.1 Viewing File Content

```
cat file.txt           # print whole file
less file.txt           # paginated viewer (q to quit, / to search)
more file.txt             # simpler pager
head -n 20 file.txt         # first 20 lines
tail -n 20 file.txt          # last 20 lines
tail -f /var/log/syslog       # follow a file live (great for logs)
wc -l file.txt                  # count lines
file file.txt                    # detect file type
```

7.2 Finding Files

```
find /home -name "*.log"                # find by name
find / -type f -size +100M              # files larger than 100MB
find . -mtime -7                         # modified in the last 7 days
find . -name "*.tmp" -delete             # find and delete
locate filename.txt                       # fast search using a prebuilt index
↪  (updatedb)
which python3                               # show the path of an executable
whereis bash                                 # show binary, source, and man page
↪  locations
```

7.3 Searching File Contents

```
grep "error" logfile.txt              # search for a pattern
grep -i "error" logfile.txt            # case-insensitive
grep -r "TODO" ./src                    # recursive search through a directory
grep -n "error" file.txt                 # show line numbers
grep -v "debug" file.txt                  # invert match (show non-matching lines)
grep -E "error|warning" file.txt           # extended regex (multiple patterns)
```

7.4 Archiving and Compression

```
tar -cvf archive.tar folder/         # create a tar archive
tar -xvf archive.tar                  # extract a tar archive
tar -czvf archive.tar.gz folder/       # create gzip-compressed archive
tar -xzvf archive.tar.gz                # extract gzip-compressed archive
tar -cjvf archive.tar.bz2 folder/        # bzip2 compression (better ratio, slower)
zip -r archive.zip folder/                # create a zip
unzip archive.zip                          # extract a zip
gzip file.txt                               # compress a single file to file.txt.gz
gunzip file.txt.gz                             # decompress
```

CHAPTER 8

File Permissions and Ownership

Every file and directory has an owner, a group, and a set of permission bits for three categories: owner, group, and others.

```
-rwxr-xr--  1 alice  devs  4096 Jun 30 10:00 script.sh
```

Breaking this down: - First character: file type (`-` regular file, `d` directory, `l` symlink, `c`/`b` device files). - Next 9 characters in groups of 3: owner / group / others permissions, each `r` (read), `w` (write), `x` (execute) or `-` (absent).

8.1 Changing Permissions

```
chmod u+x script.sh        # add execute permission for the owner
chmod g-w file.txt           # remove write permission for the group
chmod o=r file.txt             # set "others" permission to read-only
chmod 755 script.sh              # symbolic-to-octal: rwxr-xr-x
chmod -R 644 folder/               # recursively apply to all files
```

8.2 Octal Permission Reference

Octal	Permission
7	rwx (read+write+execute)
6	rw-
5	r-x
4	r–
0	—

A common pattern, `chmod 755`, means: owner gets rwx (7), group gets r-x (5), others get r-x (5) — typical for executable scripts and directories. `chmod 644` (rw-r–r–) is typical for regular data files.

8.3 Changing Ownership

```
sudo chown alice file.txt              # change owner
sudo chown alice:devs file.txt          # change owner and group
sudo chown -R alice:devs folder/         # recursive
sudo chgrp devs file.txt                  # change group only
```

8.4 Special Permissions

- **SUID** (`chmod u+s`): when set on an executable, it runs with the file owner's privileges rather than the invoking user's (classic example: `/usr/bin/passwd`, which needs root privileges to edit `/etc/shadow`).
- **SGID** (`chmod g+s`): on a directory, new files inherit the directory's group instead of the creator's primary group — useful for shared team directories.
- **Sticky bit** (`chmod +t`): on a directory (classic example: `/tmp`), only the file's owner (or root) can delete or rename files inside it, even if others have write access to the directory.

```
chmod u+s /path/to/binary
chmod g+s /shared/teamdir
chmod +t /shared/public_uploads
```

8.5 umask

`umask` defines the default permissions subtracted when new files/directories are created. The default `umask 022` results in new files getting `644` and new directories `755`.

```
umask          # view current umask
umask 027      # set a stricter default
```

CHAPTER 9

Standard Streams, Redirection, and Pipes

Every process has three standard streams:

Stream	Number	Default
stdin	0	keyboard input
stdout	1	terminal output
stderr	2	terminal error output

9.1 Redirection

```
command > file.txt          # redirect stdout to a file (overwrite)
command >> file.txt          # redirect stdout, append
command 2> errors.txt          # redirect stderr only
command > out.txt 2>&1          # redirect both stdout and stderr to the same file
command < input.txt               # use a file as stdin
command &> all_output.txt          # shorthand for redirecting both streams (bash)
command 2>/dev/null                  # discard error output entirely
```

9.2 Pipes

The pipe | sends the stdout of one command directly into the stdin of the next, enabling powerful command chains:

```
ps aux | grep nginx                    # filter process list
cat access.log | sort | uniq -c | sort -rn | head     # top repeated lines
ls -la | wc -l                          # count files in a directory
history | grep ssh                       # search command history
```

CHAPTER 10

Text Processing Power Tools

10.1 grep — pattern searching

```
grep -i "error" file.log          # case-insensitive
grep -c "error" file.log           # count matches
grep -A 3 "Exception" file.log      # show 3 lines After match
grep -B 3 "Exception" file.log       # show 3 lines Before match
grep -E "[0-9]{3}-[0-9]{4}" file      # extended regex
grep -o "user=[a-z]*" access.log      # print only the matched part
```

10.2 sed — stream editor (find & replace, line editing)

```
sed 's/foo/bar/' file.txt              # replace first occurrence per line
sed 's/foo/bar/g' file.txt              # replace all occurrences per line
sed -i 's/foo/bar/g' file.txt            # edit file in place
sed -n '5,10p' file.txt                   # print only lines 5-10
sed '/^#/d' file.txt                       # delete comment lines
```

10.3 awk — pattern scanning and field processing

awk treats each line as a record split into fields ($1, $2, … $NF for the last field).

```
awk '{print $1, $3}' file.txt              # print columns 1 and 3
awk -F, '{print $2}' data.csv                # use comma as field separator
awk '{sum += $2} END {print sum}' data.txt    # sum a column
awk '$3 > 100 {print $0}' data.txt              # filter rows by condition
awk 'NR==1,NR==10' file.txt                       # print lines 1 through 10
```

10.4 Sorting and Deduplication

```
sort file.txt              # alphabetical sort
sort -n file.txt             # numeric sort
sort -r file.txt              # reverse order
sort -k2 file.txt               # sort by 2nd column
sort -u file.txt                  # sort and remove duplicates
uniq file.txt                       # remove adjacent duplicate lines (input must
 ↪  be sorted)
uniq -c file.txt                      # count occurrences of each line
```

10.5 Cutting and Joining Text

```
cut -d',' -f1,3 data.csv          # extract fields 1 and 3 from CSV
cut -c1-5 file.txt                   # extract characters 1-5 of each line
paste file1.txt file2.txt              # merge files line by line
tr 'a-z' 'A-Z' < file.txt                # translate lowercase to uppercase
tr -d ' ' < file.txt                       # delete spaces
column -t -s',' data.csv                     # pretty-print as a table
```

10.6 Combining Tools (Real-World Examples)

```
# Top 5 IPs hitting your web server
awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -5

# Find and kill all processes matching a name
ps aux | grep myapp | awk '{print $2}' | xargs kill -9

# Count how many .log files were modified today
find /var/log -name "*.log" -mtime -1 | wc -l

# Replace all tabs with commas in a file
sed -i 's/\t/,/g' data.tsv
```

CHAPTER 11

Vim and Nano — Terminal Text Editors

11.1 Nano (beginner-friendly)

```
nano file.txt
```

Common shortcuts (shown at the bottom of the screen, ^ means Ctrl): - `Ctrl+O` save, `Ctrl+X` exit, `Ctrl+K` cut line, `Ctrl+U` paste, `Ctrl+W` search.

11.2 Vim (powerful modal editor, ubiquitous on servers)

Vim has multiple modes: - **Normal mode** (default): navigate and issue commands. - **Insert mode** (`i`): type text. - **Visual mode** (`v`): select text. - **Command mode** (`:`): run commands like save/quit.

```
i           enter insert mode
Esc         return to normal mode
:w          save
:q          quit
:wq         save and quit
:q!         quit without saving
dd          delete current line
yy          copy (yank) current line
p           paste
u           undo
/pattern    search forward
n           repeat last search
:%s/old/new/g   replace all occurrences in the file
```

Learning at least basic Vim is essential because it is preinstalled on nearly every Linux system, while editors like nano may not be.

PART IV

Part III — Users, Processes & Packages

CHAPTER 12

User and Group Management

Linux is inherently multi-user. Each user has a numeric **UID** and belongs to one primary group plus optionally several secondary groups.

```
sudo useradd -m -s /bin/bash alice       # create user with home dir and bash shell
sudo passwd alice                          # set password
sudo usermod -aG sudo alice                 # add user to the sudo group
sudo usermod -L alice                        # lock an account
sudo userdel -r alice                       # delete user and their home directory
sudo groupadd developers                       # create a group
sudo gpasswd -a alice developers                 # add user to a group
groups alice                                       # list a user's groups
id alice                                             # show UID, GID, and group
 ↪  memberships
whoami                                                 # current username
who                                                     # currently logged-in users
last                                                        # login history
```

12.1 Key Files

- `/etc/passwd` — username, UID, GID, home directory, default shell (password hash no longer stored here).
- `/etc/shadow` — encrypted passwords and password aging policy (readable only by root).
- `/etc/group` — group definitions and memberships.
- `/etc/sudoers` — defines who can use `sudo` and with what privileges; edit safely with `visudo` (validates syntax before saving).

12.2 Switching Users and Privilege Escalation

```
su alice              # switch to another user (needs their password)
su -                    # switch to root with a full login shell
sudo command             # run a single command as root (uses your own password)
sudo -i                    # start an interactive root shell
sudo -u alice command         # run a command as a specific user
```

`sudo` is generally preferred over logging in directly as root because it provides an audit trail (`/var/log/auth.log`) and lets you grant fine-grained privileges instead of full root access.

CHAPTER 13

Process Management

A **process** is a running instance of a program. Each has a PID (process ID), a parent (PPID), an owner, and a state.

```
ps aux                  # list all running processes (BSD style)
ps -ef                   # list all processes (System V style)
top                        # live, interactive process viewer
htop                         # improved, colorized version of top (if installed)
pgrep nginx                    # find PIDs matching a name
pidof nginx                      # same idea, simpler output
```

13.1 Controlling Processes

```
kill PID                  # send SIGTERM (graceful stop request)
kill -9 PID                 # send SIGKILL (force kill, cannot be ignored)
kill -1 PID                   # send SIGHUP (often used to reload config)
killall processname             # kill by name
pkill -f "pattern"                # kill processes matching a command-line pattern
```


13.2 Common Signals

Signal	Number	Meaning
SIGHUP	1	hangup; often reload config
SIGINT	2	interrupt, same as Ctrl+C
SIGKILL	9	force kill, cannot be caught or ignored
SIGTERM	15	graceful termination request (default for `kill`)
SIGSTOP	19	pause the process
SIGCONT	18	resume a paused process

13.3 Foreground, Background, and Job Control

```
long_running_command &     # run in background
jobs                        # list background jobs
fg %1                         # bring job 1 to foreground
bg %1                           # resume job 1 in background
Ctrl+Z                            # suspend current foreground job
nohup command &                     # keep running after the terminal closes
disown                                # detach a job from the shell
```

For long tasks on remote servers, `tmux` or `screen` are preferred since they create a persistent session that survives disconnections:

```
tmux new -s mysession      # start a new named session
tmux attach -t mysession     # reattach later
Ctrl+b d                       # detach (inside tmux)
tmux ls                          # list sessions
```

13.4 Process Priority

```
nice -n 10 command            # start a process with lower priority (higher
↪  niceness)
renice -n 5 -p PID               # change priority of a running process
```

Niceness ranges from -20 (highest priority) to 19 (lowest); only root can set negative values.

13.5 Viewing Resource Usage

```
free -h                # memory usage
uptime                   # load averages and uptime
vmstat 1 5                 # virtual memory stats, 5 samples 1 second apart
iostat                       # disk I/O statistics (sysstat package)
df -h                           # disk space per filesystem
du -sh /var/log                   # size of a specific directory
lsof -i :80                         # which process is using port 80
lsof -p PID                           # which files a process has open
```

CHAPTER 14

Package Management

14.1 Debian/Ubuntu — APT

```
sudo apt update                    # refresh package index
sudo apt upgrade                     # upgrade installed packages
sudo apt install nginx                 # install a package
sudo apt remove nginx                    # remove a package, keep config
sudo apt purge nginx                       # remove package and config
sudo apt autoremove                          # remove unused dependencies
apt search keyword                             # search for a package
apt show nginx                                   # show package details
dpkg -l                                            # list installed packages
dpkg -i package.deb                                  # install a local .deb file
```

14.2 Red Hat/Fedora/CentOS — DNF (modern) / YUM (legacy)

```
sudo dnf update                  # update all packages
sudo dnf install httpd             # install a package
sudo dnf remove httpd                # remove a package
dnf search keyword                     # search
dnf info httpd                           # package details
rpm -qa                                    # list installed packages
rpm -ivh package.rpm                         # install a local .rpm
```

14.3 Arch Linux — Pacman

```
sudo pacman -Syu                # sync and update everything
sudo pacman -S packagename        # install
sudo pacman -R packagename          # remove
sudo pacman -Ss keyword               # search
```

14.4 Universal/Cross-Distro Formats

- **Snap** (`snap install`) — sandboxed, auto-updating packages from Canonical.
- **Flatpak** (`flatpak install`) — sandboxed desktop applications.
- **AppImage** — a single portable executable file, no installation needed.

14.5 Building From Source

When software is unavailable as a package:

```
./configure          # check dependencies, generate Makefile
make                    # compile
sudo make install         # install compiled binaries to the system
```

Modern projects often use alternative build systems like `cmake`, `meson`, or language-specific tools (`cargo build` for Rust, `pip install` for Python packages).

PART V

Part IV — Shell Scripting

CHAPTER 15

Bash Scripting Fundamentals

A shell script is a text file containing a sequence of commands, executed top to bottom.

```
#!/bin/bash
# This is a comment
echo "Hello, World!"
```

The first line, `#!/bin/bash` (the "shebang"), tells the system which interpreter to use. Make the script executable and run it:

```
chmod +x script.sh
./script.sh
# or without execute permission:
bash script.sh
```

15.1 Variables

```
name="Alice"
echo "Hello, $name"
echo "Hello, ${name}!"          # braces avoid ambiguity when concatenating
readonly PI=3.14                 # constant
unset name                         # delete a variable
```

There is no space around `=` when assigning. Variables are untyped strings by default.

15.2 Command Substitution and Arithmetic

```
current_date=$(date +%F)
echo "Today is $current_date"

count=$(ls | wc -l)

result=$((5 + 3))
echo $result                       # 8

let "x = 10 * 2"
echo $x                            # 20
```

15.3 User Input and Arguments

```
read -p "Enter your name: " name
echo "Hi, $name"

# Script arguments
echo "Script name: $0"
echo "First arg: $1"
echo "All args: $@"
echo "Number of args: $#"
```

15.4 Conditionals

```
if [ "$1" == "start" ]; then
    echo "Starting..."
elif [ "$1" == "stop" ]; then
    echo "Stopping..."
else
    echo "Usage: $0 {start|stop}"
fi
```

15.5 Common Test Operators

Test	Meaning
`-eq`, `-ne`, `-lt`, `-gt`, `-le`, `-ge`	numeric comparisons
`==`, `!=`	string comparison
`-z "$str"`	string is empty
`-n "$str"`	string is not empty
`-f file`	file exists and is a regular file
`-d dir`	directory exists
`-e path`	path exists (any type)
`-x file`	file is executable

Modern Bash also supports the more flexible `[[ ... ]]` test syntax, which handles pattern matching and avoids common quoting pitfalls:

```
if [[ $name == A* ]]; then
    echo "Name starts with A"
fi
```

15.6 Loops

```
for i in 1 2 3 4 5; do
    echo "Number: $i"
done
```

```
for file in /var/log/*.log; do
    echo "Found: $file"
done

count=0
while [ $count -lt 5 ]; do
    echo "Count: $count"
    count=$((count + 1))
done

until [ $count -eq 0 ]; do
    echo $count
    count=$((count - 1))
done
```

15.7 Functions

```
greet() {
    local name=$1
    echo "Hello, $name!"
}

greet "Alice"

# Functions returning a status / value
is_even() {
    if (( $1 % 2 == 0 )); then
        return 0    # success/true
    else
        return 1    # failure/false
    fi
}

if is_even 4; then
    echo "Even"
fi
```

15.8 Arrays

```
fruits=("apple" "banana" "cherry")
echo "${fruits[0]}"          # apple
echo "${fruits[@]}"           # all elements
echo "${#fruits[@]}"           # array length

for fruit in "${fruits[@]}"; do
    echo "$fruit"
done
```

15.9 Case Statements

```
case "$1" in
    start)
        echo "Starting service"
        ;;
    stop)
        echo "Stopping service"
        ;;
    restart)
        echo "Restarting service"
        ;;
    *)
        echo "Usage: $0 {start|stop|restart}"
        ;;
esac
```

15.10 Exit Codes and Error Handling

```
set -e                # exit immediately if any command fails
set -u                 # treat unset variables as errors
set -o pipefail          # fail if any command in a pipeline fails
set -euo pipefail         # combine all three — best practice for production scripts

command || echo "command failed"
command && echo "command succeeded"

echo $?                   # exit code of the last command (0 = success)
exit 1                      # exit the script with a specific code
```

15.11 Practical Script: Backup with Logging

```
#!/bin/bash
set -euo pipefail

SRC="/home/alice/projects"
DEST="/backups"
DATE=$(date +%Y%m%d_%H%M%S)
LOGFILE="/var/log/backup.log"

mkdir -p "$DEST"

if tar -czf "$DEST/backup_$DATE.tar.gz" "$SRC"; then
    echo "$(date): Backup succeeded: backup_$DATE.tar.gz" >> "$LOGFILE"
else
    echo "$(date): Backup FAILED" >> "$LOGFILE"
    exit 1
fi
```

```bash
# Delete backups older than 7 days
find "$DEST" -name "backup_*.tar.gz" -mtime +7 -delete
```

15.12 Cron — Scheduling Scripts

```bash
crontab -e              # edit the current user's cron jobs
crontab -l              # list current cron jobs
```

Cron format:

```
* * * * * command
| | | | |
| | | | └── day of week (0-6, Sun=0)
| | | └──── month (1-12)
| | └────── day of month (1-31)
| └──────── hour (0-23)
└────────── minute (0-59)
```

```
0 2 * * *  /home/alice/backup.sh                  # every day at 2:00 AM
*/15 * * * * /usr/local/bin/healthcheck.sh           # every 15 minutes
0 0 1 * *  /scripts/monthly_report.sh             # first day of every month
```

For modern systemd-based systems, **systemd timers** are an increasingly preferred alternative to cron, offering better logging integration and dependency handling — covered in Chapter 17.

PART VI

Part V — Networking

CHAPTER 16

Linux Networking Fundamentals

16.1 Viewing Network Configuration

```
ip addr show              # show IP addresses (modern, replaces ifconfig)
ip a                        # shorthand
ip link show                  # show network interfaces
ip route show                   # show routing table
hostname -I                       # quick way to see this machine's IPs
cat /etc/resolv.conf                # DNS resolver configuration
```

The older `ifconfig` and `route` commands (from `net-tools`) still appear in many tutorials but are deprecated in favor of the `ip` command (from `iproute2`) on modern distributions.

16.2 Configuring Interfaces

```
sudo ip addr add 192.168.1.50/24 dev eth0      # assign an IP (temporary)
sudo ip link set eth0 up                          # bring interface up
sudo ip route add default via 192.168.1.1            # set default gateway
```

For persistent configuration, edit distro-specific files: `/etc/netplan/*.yaml` (Ubuntu, using Netplan), `/etc/network/interfaces` (older Debian), or use `nmcli`/`nmtui` (NetworkManager, common on desktops and RHEL family):

```
nmcli device status
nmcli connection show
sudo nmcli connection modify "Wired connection 1" ipv4.addresses 192.168.1.50/24
nmtui                 # text-based UI for network configuration
```

16.3 Testing Connectivity

```
ping google.com                   # check basic reachability
ping -c 4 8.8.8.8                   # send only 4 packets
traceroute google.com                 # show the path packets take
mtr google.com                         # combines ping + traceroute, live updating
dig google.com                           # detailed DNS lookup
nslookup google.com                        # simpler DNS lookup
host google.com                              # quick DNS lookup
```

16.4 Ports and Sockets

```
ss -tuln                # list listening TCP/UDP sockets (modern, replaces netstat)
ss -tunap                   # include process names (needs sudo for full info)
netstat -tulnp                # older equivalent, may need installation
lsof -i :443                    # find what's using port 443
nc -zv host 22                    # check if a specific port is open (netcat)
nc -l -p 8080                       # listen on a port (quick test server)
```

16.5 Transferring Files Over the Network

```
scp file.txt user@remote:/path/             # secure copy to a remote server
scp -r folder/ user@remote:/path/              # copy a directory
scp user@remote:/path/file.txt .                 # copy from remote to local
rsync -avz src/ user@remote:/dest/                  # efficient sync, only
↪  transfers diffs
rsync -avz --delete src/ dest/                        # mirror, deleting extras at
↪  destination
wget https://example.com/file.zip                        # download a file
curl -O https://example.com/file.zip                        # download with curl
curl -I https://example.com                                   # fetch headers only
curl -X POST -d "key=value" https://api.example.com/endpoint     # send POST data
```

16.6 SSH — Secure Shell

```
ssh user@remote_host                # connect to a remote machine
ssh -p 2222 user@remote_host           # specify a custom port
ssh-keygen -t ed25519                    # generate a modern SSH keypair
ssh-copy-id user@remote_host               # install your public key on a remote
↪  server
ssh -i ~/.ssh/mykey user@remote_host             # use a specific private key
```

`~/.ssh/config` lets you define shortcuts:

```
Host myserver
    HostName 203.0.113.10
    User alice
    Port 2222
    IdentityFile ~/.ssh/mykey
```

Then simply: `ssh myserver`.

Best practice: disable password authentication and rely on key-based auth only, by setting `PasswordAuthentication no` in `/etc/ssh/sshd_config` and restarting `sshd`.

16.7 Firewalls

UFW (Uncomplicated Firewall, Ubuntu-friendly front-end for iptables/nftables):

```
sudo ufw enable
sudo ufw allow 22/tcp
sudo ufw allow from 192.168.1.0/24 to any port 3306
sudo ufw deny 80/tcp
sudo ufw status verbose
```

firewalld (common on RHEL/CentOS/Fedora):

```
sudo firewall-cmd --add-service=http --permanent
sudo firewall-cmd --add-port=8080/tcp --permanent
sudo firewall-cmd --reload
sudo firewall-cmd --list-all
```

iptables / nftables (lower-level, underlying both of the above):

```
sudo iptables -L -n -v                          # list current rules
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
sudo iptables -A INPUT -j DROP
```

16.8 DNS and /etc/hosts

```
cat /etc/hosts
```

```
127.0.0.1   localhost
192.168.1.100  myserver.local  myserver
```

Entries here override DNS lookups for the listed names — useful for local testing and quick overrides.

16.9 Network Troubleshooting Workflow

1. Check physical/link layer: `ip link show` (is the interface “UP”?).
2. Check IP assignment: `ip addr show`.
3. Check local routing: `ip route show`.
4. Check name resolution: `dig` or `getent hosts`.
5. Check reachability: `ping` the gateway, then a public IP, then a domain name (isolates DNS vs. connectivity issues).
6. Check the specific service/port: `ss -tuln`, `curl`, or `telnet host port`.
7. Check firewall rules on both ends.

PART VII

Part VI — System Administration

CHAPTER 17

systemd — Services, Targets, and Timers

systemd is the init system and service manager used by most modern distributions (Ubuntu, Debian, RHEL/Fedora, Arch, openSUSE). It manages services (“units”), parallelizes startup, and provides dependency-based ordering.

17.1 Managing Services

```
sudo systemctl start nginx           # start a service now
sudo systemctl stop nginx              # stop a service
sudo systemctl restart nginx             # restart
sudo systemctl reload nginx                # reload config without dropping
↪  connections
sudo systemctl enable nginx                   # start automatically at boot
sudo systemctl disable nginx                    # don't start at boot
systemctl status nginx                            # check current status
systemctl is-active nginx                           # quick active/inactive check
systemctl is-enabled nginx                           # quick enabled/disabled check
systemctl list-units --type=service                      # list all loaded service
↪  units
systemctl list-unit-files                                  # list all unit files and
↪  their state
```

17.2 Creating a Custom Service

`/etc/systemd/system/myapp.service`:

```
[Unit]
Description=My Custom Application
After=network.target

[Service]
Type=simple
User=alice
WorkingDirectory=/home/alice/myapp
ExecStart=/usr/bin/python3 /home/alice/myapp/app.py
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload      # reload systemd's unit definitions
sudo systemctl enable --now myapp.service
```

17.3 systemd Timers (cron alternative)

`/etc/systemd/system/backup.timer`:

```
[Unit]
Description=Run backup daily

[Timer]
OnCalendar=daily
Persistent=true

[Install]
WantedBy=timers.target
```

Paired with a matching `backup.service` that defines `ExecStart=/path/to/backup.sh`. Enable with `systemctl enable --now backup.timer`. List active timers with `systemctl list-timers`.

17.4 Targets (Runlevel Equivalents)

```
systemctl get-default                    # show default boot target
sudo systemctl set-default multi-user.target   # boot to CLI, no GUI
sudo systemctl isolate rescue.target          # switch to single-user/rescue mode
↪  now
```


Common targets: `poweroff.target`, `rescue.target`, `multi-user.target` (CLI, networked), `graphical.target` (GUI), `reboot.target`.

CHAPTER 18

Logging

18.1 journalctl (systemd's journal)

```
journalctl                      # view all logs
journalctl -u nginx               # logs for a specific service
journalctl -f                       # follow logs live (like tail -f)
journalctl --since "1 hour ago"       # filter by time
journalctl -p err                       # filter by priority (err, warning, etc.)
journalctl -b                             # logs since the current boot
journalctl --disk-usage                     # how much space the journal is using
sudo journalctl --vacuum-size=200M            # shrink the journal
```

18.2 Traditional Log Files

File	Contents
`/var/log/syslog` or `/var/log/messages`	general system messages
`/var/log/auth.log` or `/var/log/secure`	authentication and sudo events
`/var/log/kern.log`	kernel messages
`/var/log/dmesg`	boot-time kernel ring buffer
`/var/log/apt/` or `/var/log/yum.log`	package manager history
`/var/log/nginx/`, `/var/log/apache2/`	web server access/error logs

18.3 Log Rotation

`logrotate` (configured in `/etc/logrotate.conf` and `/etc/logrotate.d/`) prevents logs from growing forever:

```
/var/log/myapp/*.log {
    daily
    rotate 14
    compress
    missingok
    notifempty
}
```

CHAPTER 19

Storage Management

19.1 Disks and Partitions

```
lsblk                          # tree view of disks and partitions
sudo fdisk -l                     # detailed partition table (MBR-focused)
sudo parted -l                      # detailed partition table (GPT-friendly)
sudo fdisk /dev/sdb                    # interactive partitioning tool (n=new,
 ↪  p=primary, w=write)
sudo mkfs.ext4 /dev/sdb1                 # format a partition with ext4
sudo mkfs.xfs /dev/sdb1                    # format with XFS
```

19.2 Mounting

```
sudo mkdir /mnt/data
sudo mount /dev/sdb1 /mnt/data
sudo umount /mnt/data
```

Add to `/etc/fstab` for persistence across reboots (see Chapter 4).

19.3 Logical Volume Manager (LVM)

LVM adds a flexible layer between raw disks and filesystems, allowing volumes to be resized, spanned across disks, and snapshotted.

```
sudo pvcreate /dev/sdb /dev/sdc           # initialize physical volumes
sudo vgcreate myvg /dev/sdb /dev/sdc         # create a volume group from them
sudo lvcreate -L 50G -n mylv myvg               # create a 50GB logical volume
sudo mkfs.ext4 /dev/myvg/mylv                     # format it
sudo lvextend -L +20G /dev/myvg/mylv                 # grow the logical volume
sudo resize2fs /dev/myvg/mylv                          # grow the ext4 filesystem to
 ↪  match
pvs; vgs; lvs                                           # quick status of each layer
```

19.4 RAID (Software, via mdadm)

```
sudo mdadm --create /dev/md0 --level=1 --raid-devices=2 /dev/sdb /dev/sdc   # RAID
 ↪  1 (mirror)
cat /proc/mdstat                                                             #
 ↪  check RAID status
sudo mdadm --detail /dev/md0
```

Common RAID levels: RAID 0 (striping, speed, no redundancy), RAID 1 (mirroring, redundancy), RAID 5 (striping with parity, needs 3+ disks), RAID 10 (mirrored stripes, fast and redundant).

19.5 Disk Usage Analysis

```
df -h                          # filesystem-level usage
du -sh /var/*                    # per-directory usage, summarized
du -h --max-depth=1 /home          # one level deep
ncdu /                               # interactive disk usage explorer (if installed)
```

CHAPTER 20

Environment and Shell Configuration

20.1 Environment Variables

```
echo $PATH                  # directories searched for executables
export MY_VAR="value"          # set a variable for this shell and child processes
env                               # list all environment variables
printenv PATH                       # print one specific variable
```

20.2 Shell Startup Files

File	When it runs
`/etc/profile`, `/etc/bash.bashrc`	system-wide, all users
`~/.bash_profile` or `~/.profile`	once at login
`~/.bashrc`	every new interactive non-login shell (most common place for aliases/functions)
`~/.bash_logout`	on logout


```
# Common ~/.bashrc additions
alias ll='ls -alh'
alias gs='git status'
export PATH="$HOME/bin:$PATH"
export EDITOR=vim
```

After editing, apply changes without restarting the shell:

```
source ~/.bashrc
# or
. ~/.bashrc
```

PART VIII

Part VII — Security

CHAPTER 21

Linux Security Fundamentals

21.1 Principle of Least Privilege

Never run software as root unless strictly necessary. Use dedicated service accounts with minimal permissions, and rely on `sudo` with granular rules rather than logging in as root directly.

21.2 Keeping Systems Updated

```
sudo apt update && sudo apt upgrade -y       # Debian/Ubuntu
sudo dnf update -y                             # RHEL/Fedora
sudo unattended-upgrades                         # automate security patches
 ↪  (Debian/Ubuntu)
```

Unpatched software is the single most common entry point for attackers — automate updates wherever feasible, especially for security patches.

21.3 SSH Hardening

In `/etc/ssh/sshd_config`:

```
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
Port 2222
AllowUsers alice bob
```

Restart with `sudo systemctl restart sshd`. Combine with `fail2ban` to automatically ban IPs after repeated failed login attempts:

```
sudo apt install fail2ban
sudo systemctl enable --now fail2ban
fail2ban-client status sshd
```

21.4 Mandatory Access Control: SELinux and AppArmor

Standard Linux permissions (discretionary access control) only restrict based on user/group. **Mandatory access control** systems add an additional, centrally-enforced policy layer that confines what each program can do, even if it's running as root.

SELinux (Red Hat family):

```
getenforce                      # check current mode (Enforcing/Permissive/Disabled)
sudo setenforce 1                    # set to Enforcing temporarily
sudo sestatus                           # detailed status
ls -Z file.txt                             # view a file's SELinux context
sudo semanage port -a -t http_port_t -p tcp 8081   # allow a service to use a new
↪ port
```

AppArmor (Debian/Ubuntu family):

```
sudo aa-status                    # show loaded profiles and their mode
sudo aa-enforce /etc/apparmor.d/usr.sbin.nginx   # enforce a profile
sudo aa-complain /etc/apparmor.d/usr.sbin.nginx     # set to complain (log-only)
↪ mode
```

21.5 File Integrity and Auditing

```
sudo apt install auditd            # detailed system call auditing
sudo auditctl -w /etc/passwd -p wa -k passwd_changes   # watch a file for
↪ writes/attribute changes
ausearch -k passwd_changes              # search audit logs by key
sudo apt install aide                      # file integrity checker (detects
↪ unexpected changes)
sudo aide --init && sudo aide --check
```

21.6 Network-Facing Hardening Checklist

- Close unused ports; only expose what's necessary (`ss -tuln`).
- Use a firewall (`ufw`/`firewalld`/`nftables`) with default-deny inbound policy.
- Disable unused services (`systemctl disable`).
- Use TLS for any service handling sensitive data (Let's Encrypt via `certbot` is free and automatable).
- Rotate credentials and use a secrets manager rather than hardcoding passwords in scripts.
- Run `lynis audit system` (a popular open-source hardening/auditing tool) for an automated security report.

21.7 Password and Account Policy

```
sudo chage -l alice                # view password aging info for a user
sudo chage -M 90 alice                # force password change every 90 days
sudo passwd -l alice                     # lock an account
```

Edit `/etc/login.defs` and `/etc/security/pwquality.conf` to enforce password complexity and aging policies system-wide.

21.8 Encrypting Data

```
gpg --gen-key                                    # generate a GPG keypair
gpg -c sensitive_file.txt                           # symmetric encryption
↪  (password-based)
gpg -e -r recipient@example.com file.txt              # encrypt to a specific
↪  recipient's public key
openssl enc -aes-256-cbc -salt -in file -out file.enc   # quick symmetric
↪  encryption via OpenSSL
```

Full-disk encryption is typically configured during install using **LUKS**:

```
sudo cryptsetup luksFormat /dev/sdb1
sudo cryptsetup open /dev/sdb1 secure_volume
sudo mkfs.ext4 /dev/mapper/secure_volume
```

PART IX

Part VIII — Advanced Topics

CHAPTER 22

Performance Monitoring and Tuning

22.1 CPU and Memory

```
top / htop                    # live overview
mpstat 1 5                      # per-CPU statistics (sysstat package)
vmstat 1                          # memory, swap, I/O, CPU snapshots over time
free -h                             # memory and swap usage
sar -u 1 5                            # historical CPU usage (sysstat)
```

A consistently high **load average** (shown by `uptime` or `top`) relative to your CPU core count indicates contention; a load average of 4 on a 4-core machine means the CPUs are fully busy on average.

22.2 Disk I/O

```
iostat -x 1                # extended I/O statistics
iotop                        # per-process I/O usage, like top for disk activity
```

22.3 Identifying Resource Hogs

```
ps aux --sort=-%cpu | head       # top CPU consumers
ps aux --sort=-%mem | head         # top memory consumers
```

22.4 Tuning Kernel Parameters (sysctl)

```
sysctl -a                           # list all current kernel parameters
sudo sysctl vm.swappiness=10           # reduce swap usage preference (temporary)
sudo sysctl -w net.ipv4.ip_forward=1      # enable IP forwarding (temporary)
```

For persistence across reboots, add entries to `/etc/sysctl.conf` or a file in `/etc/sysctl.d/`:

```
vm.swappiness=10
net.core.somaxconn=1024
```

Apply with `sudo sysctl -p`.

22.5 Resource Limits (ulimit)

```
ulimit -a                   # show all current limits for this shell
ulimit -n 4096                 # raise the open-files limit for this session
```

Persistent limits are set in `/etc/security/limits.conf`, important for database and high-connection-count servers.

CHAPTER 23

The Linux Kernel

23.1 What the Kernel Does

The kernel manages four major responsibilities: process scheduling, memory management, device drivers/hardware abstraction, and filesystem/networking I/O.

```
uname -r              # current kernel version
uname -a                # full kernel and system info
lsmod                     # list loaded kernel modules
modprobe module_name        # load a kernel module
sudo rmmod module_name        # unload a kernel module
dmesg                           # kernel ring buffer (boot and hardware messages)
dmesg | tail -50                    # most recent kernel messages
```

23.2 /proc and /sys

These virtual filesystems expose live kernel state as readable/writable "files," letting you inspect and sometimes tune the running kernel without rebooting:

```
cat /proc/cpuinfo
cat /proc/meminfo
cat /proc/loadavg
cat /proc/<pid>/status        # detailed info about a specific process
echo performance | sudo tee /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor
```

23.3 Upgrading the Kernel

Generally handled through your package manager (`apt install linux-image-generic`, `dnf update kernel`) rather than manual compilation, unless you need custom drivers or a specialized configuration.

CHAPTER 24

Containers and Virtualization

24.1 Why Containers

Containers package an application with its dependencies into an isolated, lightweight unit that shares the host kernel (unlike a full virtual machine, which virtualizes hardware and runs a separate kernel). This makes containers fast to start and efficient with resources.

24.2 Docker Basics

```
docker --version
docker pull nginx                          # download an image
docker run -d -p 8080:80 nginx               # run a container, mapping host port
 ↪  8080 to container port 80
docker ps                                       # list running containers
docker ps -a                                      # list all containers, including
 ↪  stopped
docker stop <container_id>                          # stop a container
docker rm <container_id>                              # remove a stopped container
docker images                                           # list downloaded images
docker exec -it <container_id> bash                        # open a shell inside a
 ↪  running container
docker logs <container_id>                                    # view container logs
docker build -t myapp:1.0 .                                      # build an image
 ↪  from a Dockerfile
```

A minimal `Dockerfile`:

```
FROM python:3.12-slim
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["python", "app.py"]
```

24.3 Linux Namespaces and cgroups (What Powers Containers)

Containers aren't magic — they're built from standard Linux kernel features: - **Namespaces** isolate what a process can see: PID namespace (separate process tree), network namespace (separate interfaces/IPs), mount namespace (separate filesystem view), UTS namespace (separate hostname), user namespace (separate

UID mapping). - **cgroups** (control groups) limit and account for resource usage (CPU, memory, I/O) per group of processes.

```
unshare --pid --fork --mount-proc bash      # manually create a new PID namespace,
↪  illustrating the concept
cat /sys/fs/cgroup/memory.max                  # inspect cgroup v2 memory limit for
↪  current scope
```

24.4 Virtual Machines

```
# KVM/QEMU — the standard Linux hypervisor stack
sudo apt install qemu-kvm libvirt-daemon-system virt-manager
virsh list --all                  # list VMs via libvirt
virt-install --name myvm --memory 2048 --vcpus 2 --disk size=20 --cdrom ubuntu.iso
```

KVM turns the Linux kernel itself into a type-1 hypervisor, making it the foundation for most cloud providers' virtualization (often paired with QEMU for device emulation).

CHAPTER 25

Compiling Software and Development Tools

```
sudo apt install build-essential        # gcc, make, and friends (Debian/Ubuntu)
sudo dnf groupinstall "Development Tools"   # equivalent on RHEL/Fedora

gcc -o program program.c                  # compile a C program
make                                      # run a Makefile in the current directory
git clone https://github.com/user/repo.git      # clone a repository
git status; git add .; git commit -m "msg"; git push   # basic git workflow
```

25.1 Version Managers

For language runtimes, version managers avoid permission headaches and let you switch versions per-project:

```
# Example: Node.js via nvm
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/.../install.sh | bash
nvm install --lts
nvm use 20

# Example: Python via pyenv
pyenv install 3.12.3
pyenv global 3.12.3
```

PART X

Part IX — Troubleshooting & Reference

CHAPTER 26

Systematic Troubleshooting

When something breaks, work through layers methodically rather than guessing randomly:

1. **Reproduce and isolate** — confirm the exact symptom and the conditions that trigger it.
2. **Check the logs first** — `journalctl -xe`, `journalctl -u <service>`, and `/var/log/` almost always contain the real error.
3. **Check resource exhaustion** — is the disk full (`df -h`)? Out of memory (`free -h`, `dmesg | grep -i oom`)? Out of inodes (`df -i`)?
4. **Check service state** — `systemctl status <service>` shows recent failures and exit codes directly.
5. **Check permissions** — a huge fraction of "it doesn't work" issues are simple permission or ownership mismatches.
6. **Check configuration syntax** — many services have a `--test` or `-t` flag to validate config without restarting, e.g. `nginx -t`, `sshd -t`, `apachectl configtest`.
7. **Check connectivity** if networked — work outward from the local machine (interface → gateway → DNS → remote host → specific port).
8. **Bisect changes** — what changed recently? Check `apt history`, recent file modification times (`find /etc -mtime -1`), or git history if config is version-controlled.

26.1 Common Problems and Fixes

Symptom	Likely cause	Where to look
"Permission denied"	wrong owner/permissions, or SELinux/AppArmor blocking	`ls -l`, `id`, `getenforce`, `aa-status`
Service won't start	config syntax error, port conflict, missing dependency	`systemctl status`, `journalctl -u`, config test flag
"No space left on device"	disk or inode exhaustion	`df -h`, `df -i`, `du -sh /var/*`
System extremely slow	high load, swapping, runaway process	`top`, `vmstat`, `free -h`
Can't SSH in	firewall, wrong key, sshd misconfigured, network down	`ss -tuln`, `ufw status`, `sshd -t`, `ping`
"Command not found"	not installed, or not in `$PATH`	`which`, `echo $PATH`, package manager search

Symptom	Likely cause	Where to look
Boot fails / hangs	bad fstab entry, broken initramfs, kernel panic	boot in rescue mode, check `journalctl -b -1` (previous boot)
DNS not resolving	bad `/etc/resolv.conf`, DNS server down	`dig`, `cat /etc/resolv.conf`, `systemd-resolve --status`

26.2 Rescue and Recovery

- Boot into a **GRUB recovery/rescue menu** by holding Shift (BIOS) or Esc (UEFI) at boot, then choosing "Advanced options."
- Boot to **single-user mode** for a root shell without full networking/services, useful for fixing a broken config that prevents normal boot.
- Use a **live USB** to mount a broken system's disk (`mount /dev/sdaX /mnt`) and repair files from outside the broken environment entirely.
- `fsck /dev/sdaX` checks and repairs filesystem corruption (run only on an unmounted partition).

CHAPTER 27

Master Command Cheat Sheet

27.1 File & Directory

`ls`, `cd`, `pwd`, `mkdir`, `rmdir`, `cp`, `mv`, `rm`, `touch`, `find`, `locate`, `tree`, `ln`

27.2 Viewing & Editing Text

`cat`, `less`, `more`, `head`, `tail`, `nano`, `vim`, `grep`, `sed`, `awk`, `cut`, `sort`, `uniq`, `wc`, `tr`, `diff`

27.3 Permissions & Ownership

`chmod`, `chown`, `chgrp`, `umask`, `sudo`, `su`

27.4 Users & Groups

`useradd`, `usermod`, `userdel`, `groupadd`, `passwd`, `id`, `whoami`, `who`, `last`

27.5 Processes

`ps`, `top`, `htop`, `kill`, `killall`, `pkill`, `nice`, `renice`, `jobs`, `fg`, `bg`, `nohup`

27.6 Package Management

`apt`, `dpkg`, `dnf`, `yum`, `rpm`, `pacman`, `snap`, `flatpak`

27.7 Networking

`ip`, `ping`, `traceroute`, `ss`, `netstat`, `dig`, `nslookup`, `curl`, `wget`, `ssh`, `scp`, `rsync`

27.8 Disks & Filesystems

`df`, `du`, `mount`, `umount`, `fdisk`, `parted`, `lsblk`, `mkfs`, `fsck`

27.9 System Info

`uname`, `uptime`, `free`, `lscpu`, `lsusb`, `lspci`, `dmesg`, `hostnamectl`

27.10 systemd

`systemctl`, `journalctl`, `systemd-analyze`

27.11 Archiving

`tar`, `gzip`, `gunzip`, `zip`, `unzip`

27.12 Compression Quick Reference

Command	Action
`tar -czvf out.tar.gz dir/`	compress a directory
`tar -xzvf out.tar.gz`	extract
`tar -tzvf out.tar.gz`	list contents without extracting

CHAPTER 28

Where to Go From Here

- **Practice deliberately**: spin up a free-tier cloud VM or a local VM and break things on purpose, then fix them — this builds real troubleshooting instinct far faster than reading alone.
- **Read source configs**: explore `/etc` on a real system; most config files have comments explaining each option.
- **Learn one distro deeply, then branch out**: master either the Debian/APT world or the Red Hat/DNF world first; the concepts transfer, only the tooling syntax changes.
- **Specialize**: pick a direction — DevOps/cloud (Kubernetes, Terraform, CI/CD), security (penetration testing, hardening), or systems programming (kernel, C, performance engineering).
- **Get certified (optional but useful)**: CompTIA Linux+, LPIC-1/2, or Red Hat Certified System Administrator (RHCSA) validate and structure the knowledge in this course.
- **Read the manual**: `man <command>` and the Arch Wiki (excellent even for non-Arch users) are two of the best references in existence.

Congratulations on completing the course — you now have a solid, end-to-end foundation covering Linux from first principles through production system administration.